

Analysis of Data Leakage Risks in Tor Exit Nodes via Man-in-the-Middle Attacks

Eonshik Kim

eo128494@ucf.edu

University of Central Florida
Orlando, Florida, USA

Paul Shanklin

pa466945@ucf.edu

University of Central Florida
Orlando, Florida, USA

Abstract

Tor provides anonymity through onion routing however, it does not provide confidentiality of transmitted data. The final layer of Tor encryption is stripped before traffic reaches its destination at the exit node, which leaves data fully exposed to the exit node operator. We present our proof of concept study by demonstrating this vulnerability in a controlled Docker environment using a simulated malicious exit node. Using Scapy to sniff traffic on the container's network interface. We successfully managed to intercept plaintext HTTP POST credentials transmitted through a local Tor SOCKS5 proxy and recover a username and password without any decryption needed. A comparison test using HTTPS confirmed that end-to-end encryption at the application layer prevents attackers from viewing important data. The results from our study show that Tor's anonymity and confidentiality are separate from one another. Users relying on Tor without using HTTPS remain exposed to credential harvesting at malicious exit nodes.

1 Introduction and Problem Statement

1.1 Introduction

Tor, or The Onion Router, is the most widely deployed anonymity system on the internet [5], providing those who use it anonymity and the means to avoid censorship. Tor provides anonymity through a circuit-based routing architecture that routes network traffic through

volunteer relays. The system employs onion routing, where the client encrypts data in multiple layers where each layer corresponds to a relay within the circuit. This means that each relay can only decrypt its own layer to reveal the next hop's address, similar to peeling layers of an onion which is where Tor got its name from.[1] Within an average Tor circuit there are three types of nodes: entry/guard node, middle node, and exit node. The guard node receives the traffic from the user, removes the outermost encryption layer to reveal the middle node's address, which then peels off its layer to forward the traffic to the exit node. The exit node removes the final layer of Tor encryption and delivers the underlying traffic to its destination.[4]

1.2 Problem Statement

Despite Tor's unique architecture protecting user anonymity through multi-layered encryption and circuit-based routing, a critical vulnerability exists at the network's trust boundary. While Tor ensures that no single relay can correlate a user's identity with their destination, the system's design forces complete decryption of Tor's protective layers at the exit node to deliver traffic to its final destination. This requirement creates an exploitable gap when users transmit application layer data without end-to-end encryption.

When users access websites over unencrypted HTTP rather than HTTPS, their traffic becomes fully visible in plaintext to the exit node operator. This includes credentials, session tokens, and other personal information. This represents

a crucial misunderstanding many users have about Tor. While the system provides strong anonymity guarantees, it does not inherently provide confidentiality of the transmitted data itself. Malicious actors can operate exit nodes specifically to gather sensitive information from users who mistakenly believe Tor’s anonymity protections extend to data confidentiality. Huber [2] demonstrated the scale of this risk exposure by operating a Tor exit node and collecting 9 million HTTP requests over several weeks, finding that approximately 78% of Tor users browsed without the TorButton extension, leaving them vulnerable to de-anonymization and data interception. Their analysis further revealed that over 7% of observed HTTP traffic originated from social networking sites, where plaintext requests included user identifiers that could link anonymous Tor sessions to user identities. Even passive observation of exit traffic exposed sensitive information including search queries, authentication cookies, and location data. Huber concluded that HTTPS remains the only effective countermeasure against information leakage at exit nodes. Despite this, many services at the time failed to support it. This research addresses the need to demonstrate this vulnerability in a controlled environment. Using a custom Dockerized environment to simulate a private Tor network within Docker containers, we construct malicious exit nodes capable of intercepting and analyzing plaintext HTTP traffic. By capturing real credentials and sensitive data transmitted through these nodes, we aim to provide concrete evidence of the risks associated with unencrypted application-layer protocols over Tor, ultimately highlighting the critical importance of using HTTPS and its end-to-end encryption even when using anonymity networks.

2 Related Work

2.1 Malicious Exit Relay Detection and Real-World Usage

The threat of malicious Tor exit relays intercepting user traffic is well documented through user reports and academic studies. Exit relays terminate encrypted Tor circuits and forward plaintext traffic to destination servers, placing relay operators in a prime man-in-the-middle position that allows traffic sniffing, DNS poisoning, and SSL-based attacks. Early demonstrations of this vulnerability included a 2007 incident in which a security researcher published 100 POP3 credentials captured by sniffing traffic on exit relays. Subsequent academic studies developed scanning frameworks such as *exitmap* and *HoneyConnector* to analyze exit relays for credential sniffing and active MitM behavior, identifying dozens of malicious relays. These studies revealed that Tor’s BadExit flagging mechanism is reactive rather than preventive due to how it relies on manual verification by operators. This creates a window of attack when a malicious relay begins operating and when it is excluded from circuit selection [9]. Our work seeks to simulate the attacker’s perspective within an isolated Docker environment to concretely illustrate the data leakage mechanism and its implications for unencrypted traffic rather than identifying malicious relays in the Tor network.

2.2 Malicious Relay Injection and Path Compromise

An important part of exit node attacks is how attackers ensure that a victim’s traffic passes through their controlled relay. Several studies have demonstrated that Tor’s path selection algorithm can be exploited to increase the probability or guarantee that a malicious relay is selected. Sulaiman and Zhioua [6] demonstrated that adversaries can exploit the uneven distribution of relay availability across ports to inject malicious entry and exit relay pairs by targeting underserved ports. They

accomplished this by deploying 30 pairs of compromised relays on port 25 (SMTP), which only had a small amount of legitimate relays. Through this method, they achieved a compromise ratio of 52% of constructed circuits. Their approach took advantage of Tor’s bandwidth dependent path selection by advertising high bandwidth capacities for their relays. While their work primarily focused on deanonymization through traffic correlation between entry and exit relays, relay injection is relevant to confidentiality attacks. Attackers using similar techniques to inject malicious exit relays can perform plaintext interceptions.

Tan and Xiang-Yang Li [7] introduced the Trapper Attack, which achieves control over exit relay selection through deterministic not probabilistic means. Their approach utilizes “honey relays” that use selective dos to disrupt circuits not terminating at a controlled exit node. In practice, a compromised entry guard relay issues interrupt commands to destroy circuits until the users path selection algorithm chooses a compromised exit relay. The Trapper Attack managed to achieve 100% deanonymization accuracy with almost no false positives under 40 seconds on the live Tor network. The authors developed anti-detection mechanisms such as obfuscated deployment patterns, deferred attack initiation, and collaborative filtering. These techniques achieved a 94% survival rate against the Sybilhunter detection tool, demonstrating that such attacks can persist undetected.

2.3 Traffic Correlation and Risk Assessment

Johnson[3] conducted a risk assessment of Tor’s vulnerability to realistic adversaries operating at both the relay and autonomous system levels. Using TorPS (Tor Path Simulator) and a simulation framework modeling Tor’s path selection algorithm, they demonstrated that an adversary operating a single guard and exit relay pair with 100 MiB/s of bandwidth can deanonymize 80% of all Tor user types within six months. Against a single autonomous system level adversary, users

in some geographic locations face almost 100% deanonymization within three months.

Several findings from Johnson are particularly relevant to exit node data leakage. First, they established that a bandwidth ratio of approximately 5:1 between guard and exit relays maximized the probability of circuit compromise. Second, they found that the median time until a typical user’s traffic traverses a compromised exit relay is fewer than 2.5 days, showing that even newly deployed malicious exit nodes can rapidly begin intercepting user traffic. Third, their analysis revealed that exit policies and port usage significantly affect vulnerability. Users of protocols like BitTorrent, which connects to a wide range of ports, receives substantially less protection due to the smaller pool of available exit relays for those connections.

3 Methodology

In this section, we will discuss how we conduct the experiment. We will discuss the technical details so others can easily replicate the experiment. There are four stages in the methodology of this experiment.

3.1 Structuring the lab environment

First, we set up the hardware and software environment for a successful lab experiment.

Operating system: To address the compatibility issue with the latest Tor protocol, we used “debian:sid”, which included the latest package, instead of “ubuntu :latest”.

Virtualization: To create a controlled, independent network environment, we used Docker containers.

Software Environment:

- **Tor:** 0.4.8 x version, which supports the latest relay protocol
- **Python 3:** used for executing attacker and victim environments
- **scapy:** it’s for capturing network packets or controlling libraries
- **pysocks:** used to connect to a SOCKS5 proxy connection

3.2 Configuring a Malicious Exit Node

We made some configuration changes to allow the general Tor relay to be used as an attacking node.

Network Binding: We set up Tor’s receiving address to 0.0.0.0:9050 instead of 127.0.0.1 to allow communication between a host machine and a container. It allows connection to all interfaces.

Exit Policy: We allowed all traffic to all destinations, so victims’ HTTP requests aren’t blocked, and they can access the internet.

Single-Hop Circuits: We set AllowSingleHopCircuits to 1 so traffic can go through the exit node without 3-stage routing.

3.3 Packet Sniffing and Data Gathering

Tool: We used Scapy, a Python-based packet capture tool, to monitor traffic outside the network interface eth0.

Filtering Logic: Initially, we were trying to detect HTTP request headers; however, due to buffering in Tor and Docker, we encountered packet fragmentation. To resolve this issue, we chose the raw payload scan method. It ignores the protocol header and searches for sensitive keywords such as “password” or “user” in the TCP payload.

3.4 The Process of Attack Simulation

We summarize the experimental process as illustrated in Figure 1.

- (1) **Relay Initialization:** We ran the Tor process with root privileges on the docker, and confirmed that the status reached “Bootstrapped 100%” and that the network was connected.
- (2) **Operating Sniffer:** Traffic sniffing of eth0 interface started by executing `attacker.py` from the attacker’s terminal.

- (3) **Traffic Creation:** From the host machine, we ran `victim.py` via a SOCKS5 proxy and sent an HTTP POST request containing the ID and password in plain text.
- (4) **Data Exfiltration:** We captured the packet as it passed the exit node and decoded its contents, revealing credential information and proving that confidentiality was compromised.

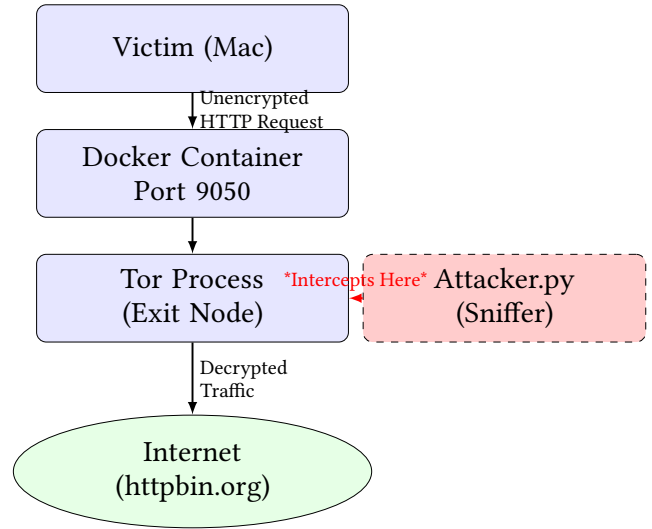


Figure 1: The flow of traffic during the attack simulation, showing where the interception occurs.

4 Evaluation & Results

This section discusses the results of the Man-in-the-Middle attack simulation in the isolated Tor environment and evaluates whether the attack was successfully carried out. The goal of the experiment was to test the hypothesis that the Tor network’s exit nodes are vulnerable because they lack encryption.

4.1 Experimental Scenario & Payload

To evaluate this, we developed `victim.py` to make an HTTP POST request via a local SOCKS5 proxy server and reach the target server `httpbin.org`.

The request contains plaintext credentials, and to prove it, we used the test password password1234. We summarize the victim's execution process as illustrated in Figure 2.

```
erik@MacBookPro project % python3 victim.py |  
[*] Request successful (200 OK)  
erik@MacBookPro project %
```

Figure 2: Sending unencrypted credentials via local Tor SOCKS5 proxy

4.2 Traffic Interception Results

During the adversary simulation, we intercepted the victim's HTTP request using the sniffer script `attacker.py` running on the malicious exit node. To be able to send data to the final destination on the TOR network's router, the exit node must disable onion routing encryption. From the experiment, when we monitored network traffic `eth0` after encryption was deactivated, we captured the target identifier "password=" and the transmitted plaintext password, and also found that all of the HTTP response data was exposed in the attacker's terminal log. We demonstrate this successful interception as illustrated in Figure 3, where all of the intercepted credential data is explicitly exposed in the attacker's terminal log.

4.3 Capture Accuracy and Efficacy

At the beginning of designing the sniffer script `attacker.py`, due to Tor or Docker packet buffering, fragmentation occurred. Therefore, some data are omitted. However, after revising the algorithm in `attacker.py`, we used the raw payload scanner, which is independent of a specific protocol layer, and successfully verified that all data was captured across 20 repeated tests. It confirms that if you have an effective sniffing technique and the attackers have an exit node, you can automate the extraction of all plaintext from that exit node.

```
erik@MacBookPro ~ % docker exec -it test2 |  
/bin/bash  
root@8825974428ee:/# python3 attacker.py  
Listening on eth0...  
[+] Credentials detected: post /post http/  
1.1  
host: httpbin.org  
user-agent: python-requests/2.32.5  
accept-encoding: gzip, deflate, zstd  
accept: */*  
connection: keep-alive  
content-length: 36  
content-type: application/x-www-form-urlencoded  
  
username=admin&password: password1234
```

Figure 3: Successful interception of plaintext credentials at the Malicious Exit Node

4.4 Implications of the Findings

The experiments demonstrated that, although the Tor network guarantees strong anonymity, it does not necessarily provide strong confidentiality, as evidenced by the data. The owner of the exit node can obtain sensitive session data by simply intercepting network traffic, without reconstructing packets or performing complex decryption.

To prove that we need end-to-end encryption, we compare it with HTTPS. As shown in Figure 4, when the victim sends data over HTTPS, the Sniffer script cannot extract any plaintext and remains in a waiting state during transmission. This evidence tells us that the owner of the exit node can see that traffic is flowing, but they cannot read the encrypted payload. In our custom Docker environment, using the unencrypted protocol over Tor shows how dangerous it is, and it ultimately shows that, even with an anonymous network, we still need to use the HTTPS protocol.

5 Discussion & Challenges

The custom Docker environment provides a controlled space that prevents us from using the real user's data and enables us to demonstrate the vulnerability of the Tor exit node. However, during

```
erik@MacBookPro ~ % docker exec -it test2  
/bin/bash  
root@8825974428ee:/# python3 attacker.py  
Listening on eth0...
```

Figure 4: Comparison test using HTTPS

the design of the malicious exit relay in the lab environment, we encountered multiple technical challenges, including limitations in the protocol version and the network.

5.1 Tor Protocol version and Consensus rejection issue

At the beginning of the project, we used the Ubuntu:latest image, but the older version (0.4.6.x) that contains this image doesn't support the essential protocol FlowCtrl and the Relay=4 setting that the Tor directory authorization server requires. Therefore, it led to a consensus rejection error. To fix this issue, we migrated to the Debian:sid image, which provided the latest package. It resolved the consensus rejection error.

5.2 Docker network and interface binding

We also encountered a network interface binding issue with Docker networking. The MAC host machine couldn't connect to the Tor SOCKS5 proxy inside a Docker container because Tor's default security setting binds to the loopback address 127.0.0.1 for SocksPorts. To resolve the issue, we manually set Tor's receiving address to 0.0.0.0:9050 by revising the torrc configuration file. In this process, to qualify Tor's strict security requirement for data directory authorization, we had to run `chown -R root:root /var/lib/tor` at the same time with root privilege access. By altering this configuration, it allowed connections from external hosts. By connecting to all interfaces, we successfully

transitioned from a regular Tor relay to an attack node.

5.3 Packet fragmentation issue

At the end, we had an issue with the design of the sniffer script, `attacker.py`. During the initial design of `attacker.py`, due to a packet fragmentation issue, we couldn't capture the sensitive data. Thus, we decided to use a raw payload scanner that captures sensitive keywords such as "password" or "username" in the TCP payload, rather than relying on a specific protocol. By adapting this scan method, we bypassed the protocol-layer limitation. We were able to find the concrete evidence about the data confidentiality threat by automating the extraction of all plaintext.

5.4 Limitation

One limitation of our custom Docker environment is that we use a single-hop circuit (AllowSingleHopCircuits 1) to simplify it. In the real Tor network, traffic is routed from guard to middle to exit via a 3-hop circuit. However, the simplification does not affect the validity of our lab result. The vulnerability we demonstrate only happens at the exit node after all the Tor encryption has been removed. Regardless of whether the traffic is single-hop or 3-hop, the plaintext application-layer data remains the same. A single hop is used only to reduce infrastructure complexity.

6 Concluding Remarks

This experiment demonstrates that Tor's anonymity protections and data confidentiality are mutually exclusive. When users transmit plaintext HTTP traffic over Tor, the exit node operator has full visibility which includes credentials and session data, without needing to perform active attacks or decryption. We reproduced the threat, despite the technical challenges with Tor protocol compatibility and packet fragmentation within a Docker environment, both of which we resolved through configuration and sniffer redesign. This demonstrates that a malicious

actor operating a single exit node can harvest sensitive data from users who believe that Tor’s anonymity protections extend to end-to-end encryption. The comparison with HTTPS traffic verifies that application layer encryption must be used regardless of the anonymity network. Future work could extend this in several directions. One example being SSL stripping attacks where HTTPS connections are reduced to HTTP before it exits the node, which represents an advanced version of our studied passive credential harvesting which relies on traffic already using HTTP. Notably, Tor Browser uses HTTPS-only mode from version 11.5, so it automatically uses HTTPS connections at startup and shows a warning message when users access an HTTP-only site[8]. It reduces the attack surface for users of the official Tor browser. However, as with our experiment’s lab results, it does not protect people who use a custom application or a SOCKS5 client. Another example would be automating the detection of credential patterns across traffic. Correlating these patterns with session data would provide a more accurate simulation of the capabilities of real malicious actors. Finally, extending the simulation to cover DNS leakage over Tor would be a natural extension as DNS requests can reveal destination information independent of HTTP traffic.

Code and Artifacts

<https://github.com/eonshikkim/tor-exit-node-mitm>

References

- [1] Roger Dingledine, Nick Mathewson, and Paul Syverson. 2004. Tor: The Second-Generation Onion Router. In *Proceedings of the 13th Conference on USENIX Security Symposium - Volume 13* (San Diego, CA) (SSYM’04). USENIX Association, Berkeley, CA, USA, 21.
- [2] Markus Huber, Martin Mulazzani, and Edgar Weippl. 2010. Tor HTTP Usage and Information Leakage. In *Proceedings of the 11th IFIP TC 6/TC 11 International Conference on Communications and Multimedia Security (CMS) (LNCS, Vol. 6109)*. Springer.
- [3] Aaron Johnson, Chris Wacek, Rob Jansen, Micah Sherr, and Paul Syverson. 2013. Users Get Routed: Traffic Correlation on Tor by Realistic Adversaries. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM.
- [4] The Onion Project. [n. d.]. Onion Routing. <https://community.torproject.org/relay/types-of-relays/>.
- [5] The Onion Project. n.d.. Tor Metrics. <https://metrics.torproject.org/>. Accessed: 2026-02-10.
- [6] Andriy Sulaiman and Sami Zhioua. 2013. Attacking Tor through Unpopular Ports. In *Proceedings of the IEEE 33rd International Conference on Distributed Computing Systems Workshops (ICDCSW)*. IEEE.
- [7] Zhi Da Henry Tan and Xiang-Yang Li. 2022. An Anonymity Vulnerability in Tor. *IEEE/ACM Transactions on Networking* 30, 6 (2022).
- [8] The Tor Project. 2022. New Release: Tor Browser 11.5. <https://blog.torproject.org/new-release-tor-browser-115/>. Accessed: 2026-04-12.
- [9] Philipp Winter, Richard Köwer, Martin Mulazzani, Markus Huber, Sebastian Schrittwieser, Stefan Lindskog, and Edgar Weippl. 2014. Spoiled Onions: Exposing Malicious Tor Exit Relays. In *Proceedings of the 14th International Symposium on Privacy Enhancing Technologies (PETS) (LNCS, Vol. 8555)*. Springer, 304–331.